

Relatório 7 - Projeto Final

Lucas Scheffer Hundsdorfer

Descrição da atividade

Para o desenvolvimento do projeto final, selecionei o conjunto de dados de ressonância magnética cerebral LGG MRI Segmentation. Este dataset é voltado para tarefas de segmentação e compreende imagens de pacientes com e sem tumores cerebrais.

A primeira tarefa aqui foi compreender os dados que eu irei trabalhar e como eles estavam separados. Primeiramente, foram coletados dados de 110 pacientes diferentes, representados em 110 pastas, cada pasta de cada paciente contém as imagens das fatias dos exames e as máscaras correspondentes, sendo que cada paciente pode ter números diferentes de fatia. Totalizando 3929 pares de imagens e máscaras ou 7858 juntando tudo. As classes entre tumor e não tumor estão divididas em 35% e 65% respectivamente, um desbalanceamento óbvio.

A divisão do dataset foi feita em 70/15/15. Dividida em pacientes, isso evita que fatias das ressonâncias fiquem em splits diferentes.

Treinamento	Validação	Teste
77 pacientes	16 pacientes	17 pacientes
2719 pares de imagens	621 pares de imagens	589 pares de imagens

O próximo passo lógico é escolher os modelos a serem treinados, mas antes eu fui atrás procurar como funciona a arquitetura dos modelos para essa área de segmentação semântica na qual eu nunca trabalhei antes. Diferente das outras tasks de visão computacional, essa é a que exige uma abordagem diferente, como pelo fato de que não é apenas mandando as imagens para o modelos e alterando os hiperparâmetros, primeiro se decide o backbone e depois uma arquitetura, é assim pelo fato de ser mais aberto então é mais fácil de alterar algo ao contrário de um “modelo fechado” e também gera mais controle sobre o todo. Até existem modelos que fazem tudo de uma vez (encoder e decoder) como o DeepLabv3+ mas eles são tecnicamente, apenas arquiteturas de Decoder que já vêm com um backbone padrão sugerido pelos autores. Então para o trabalho foi escolhido duas combinações de backbone com arquitetura diferentes:

Arquiteturas

A primeira opção foi utilizar de backbone o ResNet juntamente da arquitetura U-Net, é uma base sólida que vai juntar uma robustez e estabilidade, o ResNet sendo um dos backbones mais confiáveis e testados, já o U-Net que também não é a arquitetura mais complexa mas serve muito bem de base para análise.

A segunda opção foi de utilizar de backbone o EfficientNetB4 junto da arquitetura U-Net++, traz um pouco mais de capacidade para a extração de características e a arquitetura mais robusta que introduz sub-redes densas fazendo com que a informação do tumor seja refinada várias vezes antes de chegar à decisão.

Métricas e funções de loss

Agora a escolha das métricas além das métricas padrão mais utilizadas como acurácia, precisão, recall e F1, vão ser utilizadas outras métricas que vão dar um panorama geral dos

resultados com menos viés que essas já citadas. Vão ser utilizados também, Dice Coefficient e IoU.

- Acurácia: Vai medir o tanto que o modelo acertou baseado em todos os pixels acertados/todos pixels. Não é muito útil pois como o dataset está desbalanceado, se o modelo apenas predizer fundo para tudo ele ainda vai ter uma acurácia alta.
- Precisão: A precisão mede de tudo que o modelo predisse como tumor o quanto era tumor de fato, é uma métrica mais válida que a acurácia, já que não vai ter um viés muito alto.
- Recall: O recall vai medir de tudo que era tumor, o quanto o modelo conseguiu acertar, essa métrica é muito importante para a medicina, já que isso que vai medir a possibilidade de deixar passar um tumor que não foi visto.
- F1 Score: Essa métrica vai fazer a média entre as duas últimas duas métricas, vai fazer um balanço se o modelo está apenas predizendo tudo como tumor ou se ele está deixando tudo passar. (A métrica Dice Coefficient tem o mesmo cálculo do F1 quando se trata de segmentação binária, que é o caso.)
- IoU: Intersection over Union é a métrica mais rigorosa que vai medir o quanto do Ground Truth foi realmente acertado. É calculado a razão entre a área da interseção, onde os pixels da predição e o ground truth coincidem, pela área da união que é tudo que foi coberto por eles. Vai mostrar se o modelo consegue acertar corretamente a anatomia do tumor.

Agora passando para a metodologia do que foi feito, já foi citado a preparação do dataset, augmentation, as arquiteturas e as métricas. Ficou faltando as funções de loss utilizadas e o porquê.

Para as funções de perda foi escolhido a combinação de duas funções, Binary Cross Entropy BCE e a Dice Loss. A primeira função funciona penalizando cada pixel de acordo com a distância da previsão e do alvo, como ponto positivo vai “ensinar” para o modelo o que é fundo, o que é cérebro e o que é ruído, porém como o dataset está desbalanceado manter apenas essa função pode ter um entendimento errado das métricas. Com a junção do Dice Loss que é baseado no Dice coefficient, enquanto o coeficiente quanto mais alto melhor o loss é o contrário, quanto mais baixo melhor, já que o cálculo é feito baseado na falta de sobreposição, se faltar muito o loss sobe. Essa combinação consegue trazer uma estabilidade e uma forma de evitar que o desbalanceamento afete muito o treino.

Ambiente de desenvolvimento:

Para executar todo o projeto de hardware foi utilizado uma placa de vídeo dedicada de notebook RTX 3050 com 6GB de VRAM, 16GB de RAM.

Software e Bibliotecas:

Linguagem: Python 3.10.20.

Framework de Deep Learning: PyTorch (torch==2.11.0 / torchvision==0.26.0).

Bibliotecas Auxiliares:

segmentation_models_pytorch (v0.5.0): Para a implementação das arquiteturas U-Net (smp.Unet) e U-Net++ (smp.UnetPlusPlus) com o backbone pré-treinado EfficientNet-B4 (pesos ImageNet). A biblioteca também fornece a DiceLoss utilizada na função de perda composta. Albumentations (v2.0.8): Utilizada para a estratégia de Data Augmentation focada em preservação anatômica. OpenCV (opencv-python v4.13.0.92) e Matplotlib (v3.10.8): Para manipulação e visualização qualitativa das imagens de ressonância.

Iterações de treinamento:

Como primeira opção eu quis testar dois, um mais básico para utilizar como base e outro um pouco mais pesado para ver os possíveis resultados e a diferença. Primeiramente utilizei a arquitetura UNet junto do backbone ResNet34 como a mais simples e depois a arquitetura

UNet++ com o backbone EfficientNetB4. Esses treinamentos foram feitos com os seguintes augmentations:

- Flip Horizontal (Cérebro tem simetria aproximada entre os hemisférios).
- Rotação ($\pm 15^\circ$).
- Variação de brilho e contraste.
- Zoom leve.
- Transformação elástica leve (Simula variação anatômica).

E com os seguintes hiperparâmetros:

Hiperparâmetro	Valor	Justificativa
epochs	100	Vai ser controlado pelo early stopping
batch_size	16	Equilíbrio entre estabilidade e uso de VRAM
lr	1e-4	Padrão amplamente adotado com Adam
patience	15	Verifica se o modelo parou de aprender e para de treinar
threshold	0.5	Limiar de binarização da saída sigmoid
num_workers	4	Workers paralelos para I/O não bloquear GPU
pin_memory	True	Transferência CPU → GPU mais rápida
amp	True	Menos uso de VRAM e mais velocidade
dropout	0.2	Evitar overfitting

Os resultados desses dois modelos nessas configurações foram:

	UNet & ResNet34	Unet++ & EfficientNetB4
test_dice_tumor	0.7603	0.7629
test_iou_tumor	0.6763	0.6863
test_precision_tumor	0.7844	0.8183
test_recall_tumor	0.7743	0.7651
test_detection_acc	0.927	0.9525
val_dice	0.6116	0.6032
gap_train_val	0.2817	0.34

Ambos encoders sendo maiores do que o necessário para a base de dados trouxe um gap de val_dice para train_dice enorme, que nesse caso significa overfitting. O EfficientNetB4 sendo um encoder mais poderoso que o ResNet34 e sem weight decay e com dropout de 0.2 teve um gap ainda maior, mas mesmo ele memorizando o treino ele ainda teve um desempenho ligeiramente superior à outra combinação. Porque as representações apreendidas pelo

EfficientNet são genuinamente mais ricas mesmo overfitando mais, o que ele aprendeu sobre as features de tumor é mais discriminativo que o ResNet34. Isso aparece na precisão maior quando o EfficientNet diz "aqui é tumor", ele erra menos a localização. A contrapartida é um recall levemente pior, ele é mais conservador, perdendo um pouco mais nas bordas do tumor.

A segunda iteração de treinamento:

Como o melhor modelo tinha sido a arquitetura U++ com o backbone EfficientNetB4 obteve os melhores resultados porém estava overfitado, então tentei otimizar os hiperparâmetros junto do augmentation para reduzir o overfitting e ver os possíveis resultados, as transformações ficaram assim:

- Flip Horizontal e Vertical (Mesmo que gere imagens não correspondentes a variações clínicas o overfitting estava bem alto).
- Rotação, translação e escala ($\pm 20^\circ$, $\pm 10\%$ de deslocamento, 85–115% de escala)
- Transformação elástica e distorção em grade (simula deformações anatômicas suaves entre indivíduos diferentes).
- Zoom leve (recorte aleatório entre 85–100% da imagem)
- Variação de brilho, contraste e gama (Mais fortes comparado a primeira transformação)
- Ruído Gaussiano e blur leve (reproduz o ruído característico de imagens de MRI e variações de nitidez entre aquisições)

E aqui os hiperparâmetros para o modelo:

Hiperparâmetro	Valor	Justificativa
epochs	100	Vai ser controlado pelo early stopping
batch_size	16	Equilíbrio entre estabilidade e uso de VRAM
lr	1e-4	Padrão amplamente adotado com Adam
patience	10	Verifica se o modelo parou de aprender e para de treinar
threshold	0.5	Limiar de binarização da saída sigmoid
num_workers	4	Workers paralelos para I/O não bloquear GPU
pin_memory	True	Transferência CPU → GPU mais rápida
amp	True	Menos uso de VRAM e mais velocidade
dropout	0.3	Evitar overfitting, aumentado em 0.1
weight_decay	0.0001	Adicionado para ajudar a diminuir o overfitting.

Com essas mudanças aqui o resultado:

	Unet++ & EfficienNetB4
test_dice_tumor	0.7548
test_iou_tumor	0.6744
test_precision_tumor	0.7954
test_recall_tumor	0.7654

test_detection_acc	0.9508
val_dice	0.6059
gap_train_val	0.2812

Como visto, o gap entre o conjunto de validação e o de treino diminuiu, o overfitting desse treinamento foi realmente menor, mas os resultados na base de testes não, a única métrica que foi superada foi a recall, porém foi mínimo o ganho. O menor overfitting não resultou em melhoras nas métricas muito provavelmente pois com as novas regularizações de tornou o modelo mais conservador perdendo a capacidade de aprender padrões mais sutis.

A terceira iteração de treinamento:

Para a terceira como o overfitting estava ainda muito alto devido ao gap de dice do train e do val percebi que estava usando modelos muito grandes para um dataset não tão grande, então decidi manter as arquiteturas e testar encoders menores, mantive a família e desci os parâmetros. ResNet18 e EfficienNetB0, alterando alguns hiperparâmetros:

Hiperparâmetro	Valor	Justificativa
epochs	100	Vai ser controlado pelo early stopping
batch_size	16	Equilíbrio entre estabilidade e uso de VRAM
lr	1e-4	Padrão amplamente adotado com Adam
patience	10	Verifica se o modelo parou de aprender e para de treinar
threshold	0.5	Limiar de binarização da saída sigmoid
num_workers	4	Workers paralelos para I/O não bloquear GPU
pin_memory	True	Transferência CPU → GPU mais rápida
amp	True	Menos uso de VRAM e mais velocidade
dropout	0.3	Evitar overfitting, aumentado em 0.1
weight_decay	0.001	Adicionado para ajudar a diminuir o overfitting.
encoder_lr_factor	0.1	Preservar conhecimento pré treinado.

Então basicamente o que foi alterado foi que o weight decay aumentou para 0.001 e foi adicionado encoder_lr_factor para que consiga preservar o conhecimento pré treinado dos encoders fazendo com que tenha mais ganho nas métricas. Adicionei uma transformação a mais também para ter certeza que o overfitting ia diminuir, CoarseDropout com buracos pequenos ($\leq 32\text{px}$) mesmo que não seja anatomicamente factível eu queria diminuir o overfitting e ver o que aconteceria com os resultados:

	UNet & ResNet18	Unet++ & EfficienNetB0
test_dice_tumor	0.6812	0.7219

test_iou_tumor	0.6026	0.6388
test_precision_tumor	0.7312	0.7321
test_recall_tumor	0.6916	0.7684
test_detection_acc	0.9032	0.8896
val_dice	0.7514	0.7747
gap_train_val	0.0818	0.0594

As mudanças funcionaram para o objetivo de diminuir o overfitting. Porém ao olhar as métricas, fica algo não tão agradável já que comparado aos modelos treinados teve um desempenho menor, mas provavelmente levado ao ao fato de serem encoders com menor capacidade de aprendizado. Os modelos menores demonstraram que a regularização aplicada foi eficaz em controlar o overfitting, mas evidenciaram o trade-off inerente entre capacidade do encoder e generalização: encoders de menor porte atingem um teto de desempenho mais baixo, mesmo bem regularizados.

Quarta iteração de treinamento:

Com base nos resultados das outras iterações de treinamento, o que eu imaginei que faria sentido seria treinar o encoder EfficientNetB4 junto do decoder UNet++ com as regularizações das outras redes, já que essa combinação demonstrou as melhores métricas porém com um alto overfitting, com as regularizações das combinações que demonstraram menor overfitting o resultado esperado seria uma boa métrica com um baixo overfitting. Porém, antes de repetir o EfficientNetB4, reconsiderei o resultado da segunda iteração: mesmo com dropout, weight decay e augmentation mais agressivo, o gap treino/validação continuou em 0.28, o que sugere que o problema não era só falta de regularização, mas sim o tamanho do encoder ser grande demais para as 2700 imagens de treino disponíveis. Decidi então testar um meio-termo entre o menor encoder já treinado, EfficientNetB0, e o maior EfficientNetB4, o EfficientNetB2, mantendo a arquitetura UNet++ e exatamente os mesmos hiperparâmetros de regularização da terceira iteração dropout 0.3, weight_decay 0.001, encoder_lr_factor 0.1.

	Unet++ & EfficienNetB2
test_dice_tumor	0.7428
test_iou_tumor	0.6557
test_precision_tumor	0.7458
test_recall_tumor	0.8189
test_detection_acc	0.8829
val_dice	0.7782
gap_train_val	0.0403

Comparado a terceira iteração, o EfficientNetB2 melhorou o Dice (0.7428 vs 0.7219), o IoU (0.6557 vs 0.6388) e principalmente o recall, que chegou a 0.8189, o maior entre os seis modelos treinados neste projeto. E o gap treino/validação, em vez de aumentar com o encoder

maior, na verdade diminuiu ainda mais (0.0403 vs 0.0594 do B0). Isso confirma a hipótese: existe um ponto de equilíbrio de capacidade entre o B0 e o B4 para esse dataset, e o EfficientNetB2 combinado com as regularizações da terceira iteração encontrou esse ponto, aproveitando mais capacidade do encoder sem reintroduzir o overfitting visto no EfficientNetB4. Por esse motivo, não foi necessário repetir o treino do EfficientNetB4 regularizado como planejado inicialmente o experimento com o B2 já respondeu à pergunta de forma mais eficiente.

Dificuldades

Uma das dificuldades encontradas foi equilibrar o data augmentation em um dataset clínico. Por se tratar de imagens de MRI cerebral, optou-se inicialmente por transformações conservadoras para não comprometer a anatomia. Isso resultou em um overfitting evidente: o Dice no treino atingiu 0.91, enquanto o melhor Dice na validação foi de 0.61 uma diferença de 0.30, aumentei as transformações para verificar se o problema do overfitting era pois eram leves demais. Mesmo após as mudanças de augmentation, eu tinha escolhido dois encoders muito grandes para o dataset ResNet34 e EfficientNet-B4 são backbones com muitos parâmetros para poucas imagens. Isso fazia com que as redes memorizassem o que foi visto no treino em vez de aprender características generalizáveis, resultando em overfitting.

Outra dificuldade foi encontrar o equilíbrio entre regularização e capacidade do encoder. Ao reduzir os encoders para ResNet18 e EfficientNetB0 (terceira iteração) para conter o overfitting, o gap de fato caiu (0.08 e 0.06, respectivamente), mas o desempenho no teste caiu junto (Dice de 0.68 e 0.72, contra 0.75–0.76 dos modelos maiores), evidenciando que encoders pequenos demais têm um teto de aprendizado mais baixo para esse problema. Só ficou claro que existia um ponto de equilíbrio de capacidade depois de comparar explicitamente os resultados entre os diferentes tamanhos de encoder testados, o que motivou a quarta iteração com o EfficientNetB2.

Exemplos de Segmentação (Análise Qualitativa)

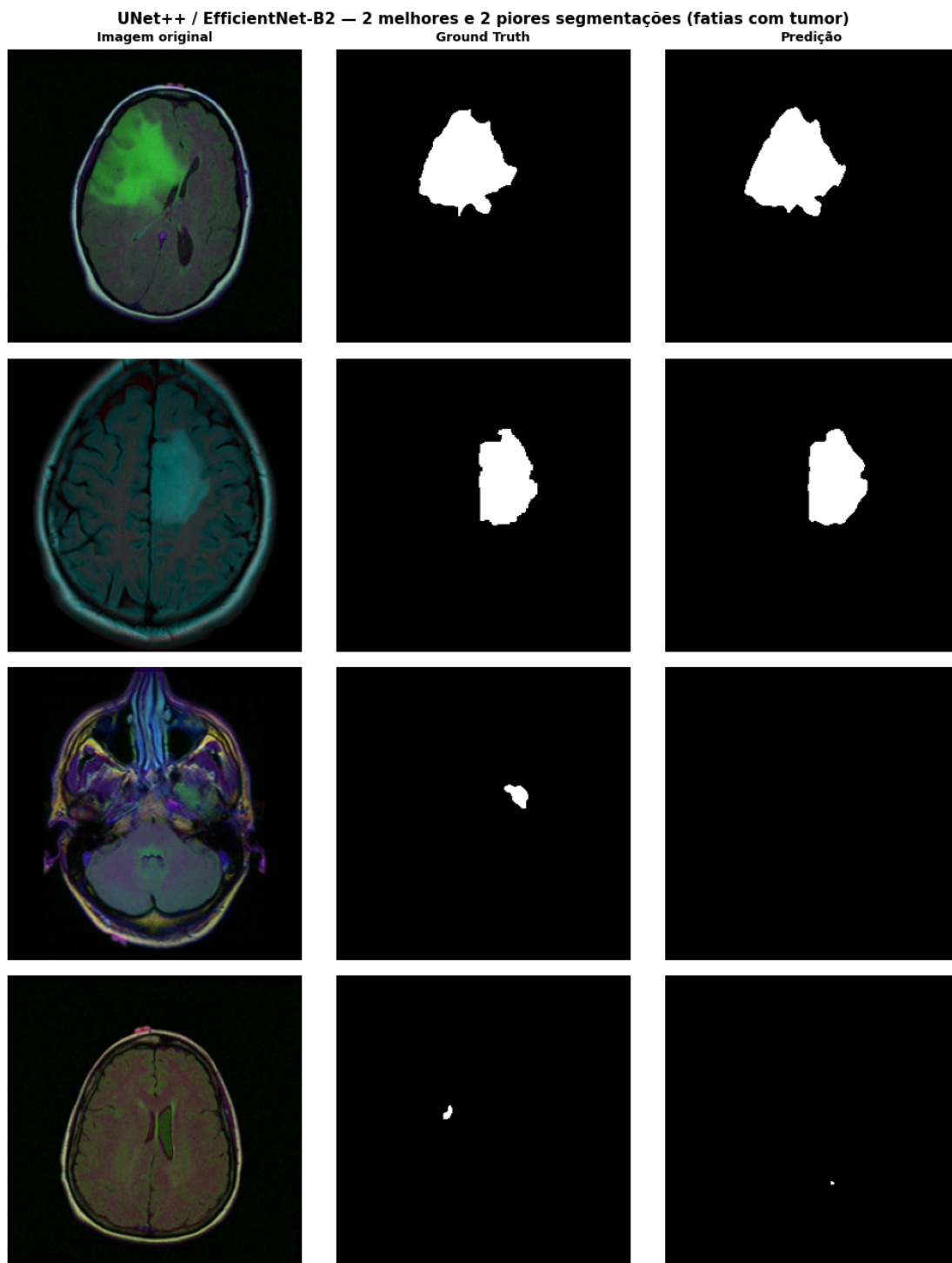


Figura 1 — UNet++ / EfficientNet-B2: 2 melhores e 2 piores segmentações (fatias com tumor) no conjunto de teste. Imagem original, ground truth e predição, da esquerda para a direita.

Nos dois melhores casos, com tumores grandes e bem definidos, a predição praticamente coincide com o ground truth. Nos dois piores casos, ambos com tumores muito pequenos (poucas dezenas de pixels), o modelo captou parcialmente um e não detectou o outro. Essa é a principal limitação qualitativa observada: tumores pequenos são consistentemente mais difíceis de segmentar do que tumores grandes, independentemente do modelo testado.

Conclusões

Ao longo das quatro iterações de treinamento, ficou evidente que o principal desafio deste projeto não foi encontrar uma arquitetura capaz de segmentar o tumor todas as combinações testadas de UNet e UNet++ com diferentes encoders chegaram a Dice acima de 0.68 nas fatias com tumor, mas sim controlar o overfitting causado pelo tamanho relativamente pequeno do dataset (2719 pares de imagem/máscara no treino) frente à capacidade de encoders pré-treinados maiores, como o ResNet34 e o EfficientNetB4.

A combinação de melhor equilíbrio do projeto foi o UNet++ com o backbone EfficientNetB2 (quarta iteração). Não é o modelo com Dice ou IoU absolutamente mais alto os modelos com encoders maiores (ResNet34 e EfficientNetB4, primeira e segunda iterações) alcançaram Dice ligeiramente superior (0.75–0.76 contra 0.74), mas apresentou de longe o menor gap treino/validação de todo o experimento (0.04, contra 0.28–0.34 dos modelos maiores) e o maior recall entre os seis modelos treinados (0.82). Em um contexto clínico, um modelo que generaliza melhor para pacientes não vistos no treino e que erra menos por "deixar passar" um tumor (falso negativo) é preferível a um modelo com poucos décimos a mais de Dice, mas fortemente ajustado ao próprio conjunto de treino.

Também vale reforçar a ressalva metodológica já levantada durante o projeto, o Dice "geral" reportado pela inferência (0.27–0.29 em todos os modelos) é enganoso, pois inclui as fatias sem tumor, onde o Dice é zero por definição matemática mesmo quando o modelo acerta perfeitamente ao não prever nada. A métrica que de fato descreve a capacidade de segmentação dos modelos é o Dice calculado apenas nas fatias com tumor presente, que foi a métrica usada para comparar as iterações ao longo deste relatório.

De forma geral, o projeto demonstrou na prática o trade-off central em segmentação semântica com datasets pequenos, aumentar a capacidade do encoder melhora a qualidade das features aprendidas, mas também aumenta o risco de overfitting, reduzir demais o encoder controla o overfitting, mas limita o teto de desempenho. A combinação de um encoder de porte médio (EfficientNetB2) com regularização adequada (dropout, weight decay e um learning rate menor para o encoder pré-treinado) foi a estratégia que melhor equilibrou esses dois lados para este dataset.

Como próximos passos, ficam como possibilidades: testar ensembles entre os modelos treinados, aplicar test-time augmentation, calibrar o limiar de binarização (atualmente fixo em 0.5) por modelo, e testar o EfficientNetB4 com a mesma receita de regularização da quarta iteração para confirmar se o problema é realmente de capacidade do encoder e não apenas de regularização insuficiente.

Referências

- Buda, M., Saha, A., & Mazurowski, M. A. (2019). Association of genomic subtypes of lower-grade gliomas with shape features automatically extracted by a deep learning algorithm. *Computers in Biology and Medicine*, 109, 218–225. (Dataset: [LGG Segmentation Dataset](https://www.kaggle.com/datasets/mateuszbuda/lgg-mri-segmentation), Kaggle)
- Ronneberger, O., Fischer, P., & Brox, T. (2015). U-Net: Convolutional Networks for Biomedical Image Segmentation. *MICCAI 2015*.
- Zhou, Z., Rahman Siddiquee, M. M., Tajbakhsh, N., & Liang, J. (2018). UNet++: A Nested U-Net Architecture for Medical Image Segmentation. *DLMIA 2018*.
- Chen, L.-C., Zhu, Y., Papandreou, G., Schroff, F., & Adam, H. (2018). Encoder-Decoder with Atrous Separable Convolution for Semantic Image Segmentation (DeepLabV3+). *ECCV 2018*.
- He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep Residual Learning for Image Recognition (ResNet). *CVPR 2016*.
- Tan, M., & Le, Q. V. (2019). EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks. *ICML 2019*.

Milletari, F., Navab, N., & Ahmadi, S.-A. (2016). V-Net: Fully Convolutional Neural Networks for Volumetric Medical Image Segmentation. (origem da Dice Loss)

Iakubovskii, P. (2019). Segmentation Models PyTorch. GitHub.
https://github.com/qubvel/segmentation_models_pytorch

Buslaev, A., Iglovikov, V. I., Khvedchenya, E., Parinov, A., Druzhinin, M., & Kalinin, A. A. (2020). Albumentations: Fast and Flexible Image Augmentations. Information, 11(2), 125.